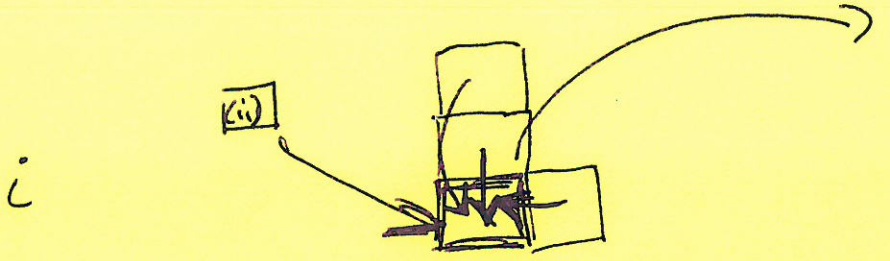
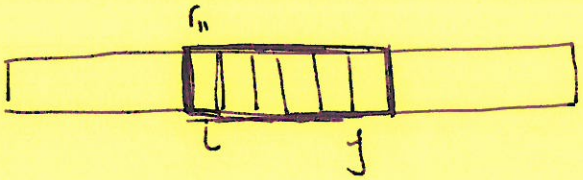
3 March 2026

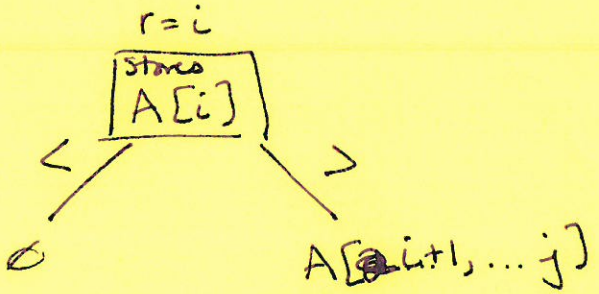


square (i,j)
rep. input array
A[i.... j]
+ Solving that
sub problem

DP:

we arrive at this by solving smaller probs first so when we get to call recursion, we just look it up!





time complexity originally

$\Theta(3^n)$

time complexity

$\Theta((n \cdot 1)(n^2)) = \Theta(n^3)$

time per prob # subprobs

The Fib. #s:

$F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$ (*)

recursive formula!

To solve this recursion

$\Theta(\phi^n)$

↑ the golden ratio

$\phi = \frac{\sqrt{5}+1}{2} \approx 1.6$

1(a) input: $n \in \mathbb{Z}$

output: the n^{th} fib. #, where

$$F(i) := \begin{cases} 0 , & i = 0 \quad \text{"zeroth"} \\ 1 , & i = 1 \quad \text{"first"} \\ F(i-1) + F(i-2) & \text{"ith"} \end{cases} \quad (**)$$

1(b) The recursive formula is given by (*) as well as (**).

1(c) work:

$F(i-1) = F(i-2) + F(i-3)$

in general, the subproblems are:

$F(n), F(n-1), F(n-2), F(n-3) \ldots$ $F(2)$ $\underbrace{F(1)\ F(0)}_{\text{base cases}}$

↑ exhaustive list of all possible subprobs I can encounter when computing $F(n)$

```
FASTSPLITTABLE(A[1..n]):
  SplitTable[n+1] ← TRUE
  for i ← n down to 1
      SplitTable[i] ← FALSE
      for j ← i to n
          if ISWORD(i, j) and SplitTable[j+1]
              SplitTable[i] ← TRUE
  return SplitTable[1]
```

Figure 3.3. Interpunctio verborum velox

(and instructors, and textbooks) make the mistake of focusing on the table—because tables are easy and familiar—instead of the *much* more important (and difficult) task of finding a correct recurrence. As long as we memoize the correct recurrence, an explicit table isn't really necessary, but if the recurrence is incorrect, we are well and truly hosed.

Dynamic programming is *not* about filling in tables.
It's about smart recursion!

Dynamic programming algorithms are best developed in two distinct stages.

1. **Formulate the problem recursively.** Write down a recursive formula or algorithm for the whole problem in terms of the answers to smaller subproblems. This is the hard part. A complete recursive formulation has two parts:
   (a) **Specification.** Describe the problem that you want to solve recursively, in coherent and precise English—not *how* to solve that problem, but *what* problem you're trying to solve. Without this specification, it is impossible, even in principle, to determine whether your solution is correct.
   (b) **Solution.** Give a clear recursive formula or algorithm for the whole problem in terms of the answers to smaller instances of *exactly* the same problem.

2. **Build solutions to your recurrence from the bottom up.** Write an algorithm that starts with the base cases of your recurrence and works its way up to the final solution, by considering intermediate subproblems in the correct order. This stage can be broken down into several smaller, relatively mechanical steps:
   (a) **Identify the subproblems.** What are all the different ways your recursive algorithm can call itself, starting with some initial input? For example, the argument to RECFIBO is always an integer between 0 and n.

WHAT?
- input
- output
- relationship btwn them

} overall goal

1st bite size chunk

that initial call counts as a subproblem.

3

Choose a memoization data structure. Find a data structure that can store the solution to *every* subproblem you identified in step (a). This is usually *but not always* a multidimensional array.

(c) **Identify dependencies.** Except for the base cases, every subproblem depends on other subproblems—which ones? Draw a picture of your data structure, pick a generic element, and draw arrows from each of the other elements it depends on. Then formalize your picture.

(d) **Find a good evaluation order.** Order the subproblems so that each one comes *after* the subproblems it depends on. You should consider the base cases first, then the subproblems that depends only on base cases, and so on, eventually building up to the original top-level problem. The dependencies you identified in the previous step define a partial order over the subproblems; you need to find a linear extension of that partial order. ***Be careful!***

(e) **Analyze space and running time.** The number of distinct subproblems determines the space complexity of your memoized algorithm. To compute the total running time, add up the running times of all possible subproblems, *assuming deeper recursive calls are already memoized*. You can actually do this immediately after step (a).

(f) **Write down the algorithm.** You know what order to consider the subproblems, and you know how to solve each subproblem. So do that! If your data structure is an array, this usually means writing a few nested for-loops around your original recurrence, and replacing the recursive calls with array look-ups.

Of course, you have to prove that each of these steps is correct. If your recurrence is wrong, or if you try to build up answers in the wrong order, your algorithm won't work!

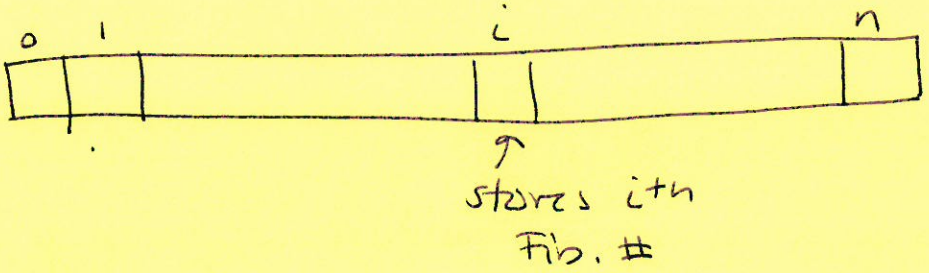
3.5 Warning: Greed is Stupid

If we're incredibly lucky, we can bypass all the recurrences and tables and so forth, and solve the problem using a *greedy* algorithm. Like a backtracking algorithm, a greedy algorithm constructs a solution through a series of decisions, but it makes those decisions directly, *without* solving at any recursive subproblems. While this approach seems very natural, it almost never works; optimization problems that can be solved correctly by a greedy algorithm are quite rare. Nevertheless, for many problems that should be solved by backtracking or dynamic programming, many students' first intuition is to apply a greedy strategy.

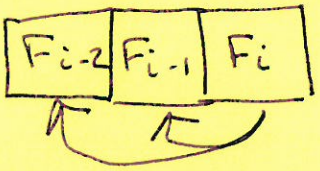
For example, a greedy algorithm for the text segmentation problem might find the shortest (or, if you prefer, longest) prefix of the input string that is

4

1(d). A 1-d array of size n+1



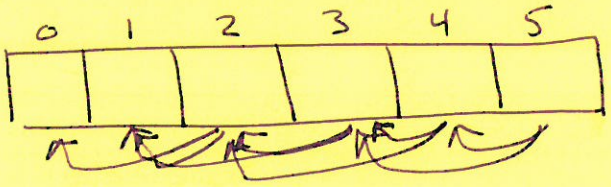
1(e). Look at ith:



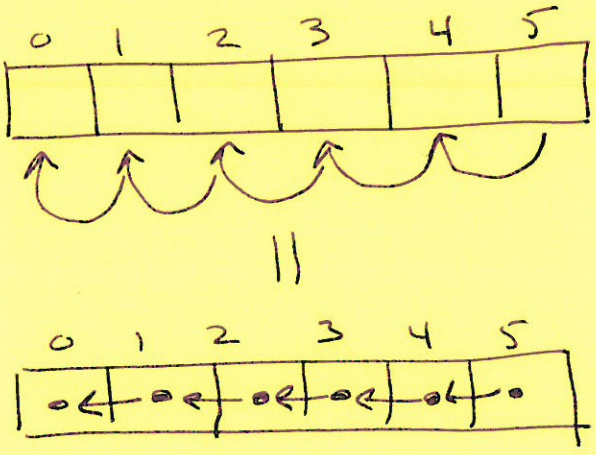
Note:

means "b relies on a" here

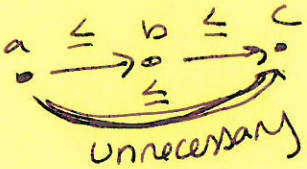
Draw all of them for F_5:



} complex ~~structure~~ digraph G



} Hasse dgm H(G)



$a \le b,\ b \le c \Rightarrow a \le c$

1(d). Evaluation order

We solve left to right
F_0 to F_n

1(e) Space: $\Theta(n)$

Time: For each $\Theta(n)$ cell, 2 lookups
= const!

$\Theta(n \cdot 1) = \Theta(n)$.

1(f).

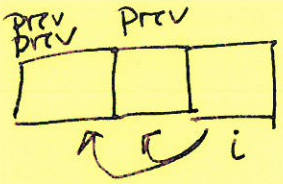
~~Fib(n)~~
~~if n=0~~
~~Fib[0] = 0~~
~~return 0~~
~~else if n=1~~
~~return 1~~
~~end if~~
~~for i=2 ... n~~

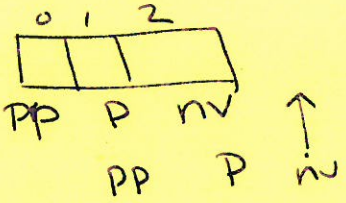
Fib(n)

SolnArray ← an array of size $n+1$
(index 0 ... n)

if $n \geq 0$
  SolnArray[0] = 0
if $n \geq 1$
  SolnArray[1] = 1
for i = 2 ... n
  SolnArray[i]
  = SolnArray[i-1]
  + SolnArray[i-2]
end for
return SolnArray[n]

I've already calculated these, so this is OK to do!






```
LOW MEMORY FIB (n)
    if n ≤ 1                                        } Θ(1)
    |  return n-1                                  }
    end if                                          }
    prevprev = 0                                    }
    prev = 1                                        }
    i = 2
    ~~nextval ← will be our sol'n: cur. NAN~~
    while i ≤ n                                     } n loops × Θ(1) inside = Θ(n)
    |   temp
    |   ~~nextval~~ ← prev + prevprev                } Θ(1)
    |   ~~prevprev ← nextval~~                       }
    |   prevprev ← prev                             }
    |   prev ← ~~nextval~~ temp                     }
    |   i++                                         }
    end while
Θ(1) return nextval
```

$\Theta(1) + \Theta(n \cdot 1) + \Theta(1) = \Theta(n)$ worst-case runtime

$\Theta(1)$ memory

related problem:

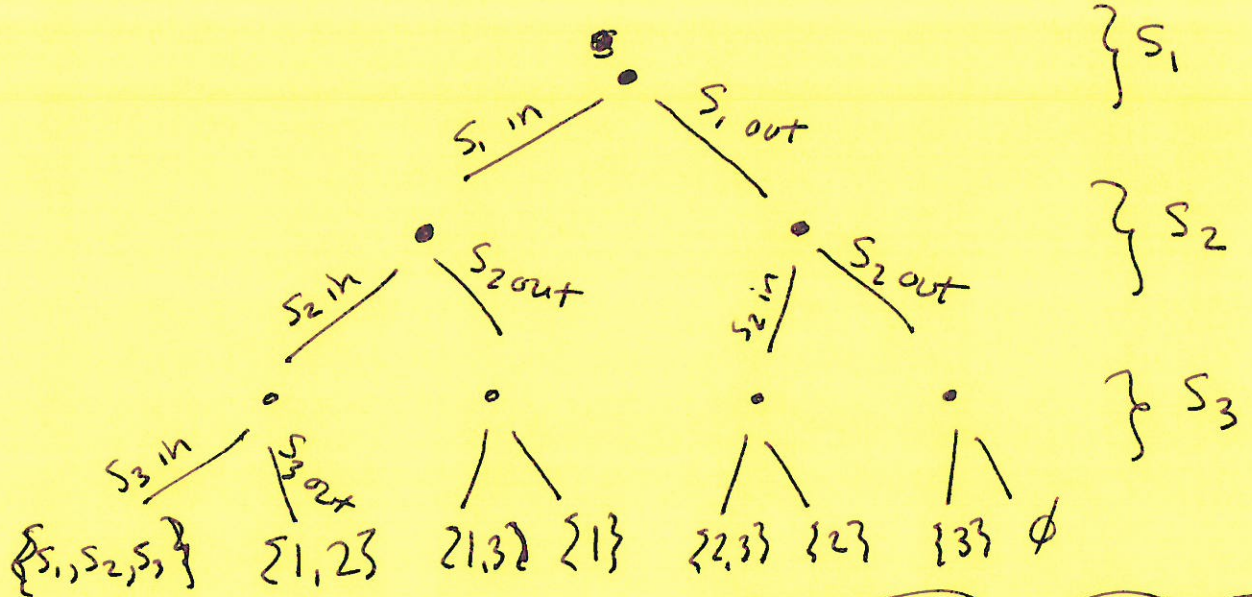
iterating through all possible subsets.

S = a set of size $n = \{s_1, s_2, s_3, \ldots, s_n\}$

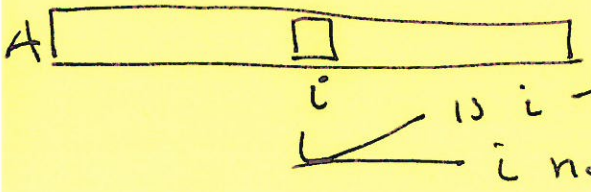
$\mathcal{P}(S)$ = the power set of $S =: 2^S$

$|\mathcal{P}(S)| = 2^n$, each elt can be in it or not * indep. of other elts

$$\underbrace{2 \times 2 \times 2 \times 2 \times 2}_{n \text{ slots}} = 2^n$$



Search Tree



Key idea:
select element.
Mark as special
Yes?
NO?
What 2 do either way.